```
begin
    using Oscar
    using Latexify
end
```

```
Matrix space of 2 rows and 2 columns
  over integer ring
```

```
begin
    # Global Mathematical Domain Definition
    Z = ZZ

    # Power Series Ring for Generating Functions (up to precision 100)
    R_ps, x_ps = power_series_ring(QQ, 100, "x_ps")

    # Linear Algebra Matrix Space over Integers (2x2 matrices)
    M_space = matrix_space(Z, 2, 2)
end
```

Lecture 8: Linear Recurrences and Fibonacci Numbers

A linear recurrence defines a sequence where each term is a linear combination of previous terms. The most famous is the **Fibonacci Sequence**: $F_n = F_{n-1} + F_{n-2}$ with base cases $F_0 = 0, F_1 = 1$.

Approach 1: Pure FP (Stateless Reduction)

Naïve recursion is computationally disastrous ($O(2^n)$). Instead, we model the sequence as a pure functional `fold` (reduce) over a tuple state (F_{n-1}, F_n). We advance the state linearly without mutable loops.

Pure FP Output (F_{50}): 12586269025

```
begin
    # Pure stateless transformation: (a, b) -> (b, a+b)
    # We fold over a dummy range 1:n to apply the transformation statelessly.
    fib_fp(n::Int=50) = n == 0 ? Z(0) : reduce((state, _) -> (state[2], state[1] + state[2]), 1:(n-1); init=(Z(0), Z(1)))[2]

    ans_fib_fp = fib_fp(50)

    Markdown.parse("""
    **Pure FP Output (\$F_{50}\$):** $(latexify("$(ans_fib_fp)", env=:inline))
    """)
end
```

Approach 2: Abstract Algebra (Generating Functions)

We shift to continuous algebra. The Fibonacci sequence can be exactly represented as the coefficients of the closed-form generating function: $G(x) = \frac{x}{1-x-x^2}$

Abstract Algebra Output (G.F. Coefficient of x^{50}): 12586269025

```
begin
    function fib_aa(n::Int=50)
        # Define the generating function in the continuous Power Series Ring
        G_x = x_ps * inv(1 - x_ps - x_ps^2)
        # The n-th Fibonacci number emerges natively as the n-th coefficient
        return Z(coeff(G_x, n))
    end

    ans_fib_aa = fib_aa(50)

    Markdown.parse("""
    **Abstract Algebra Output (G.F. Coefficient of \$x^{50}\$):**
    $(latexify("$(ans_fib_aa)", env=:inline))
    """)
end
```

Approach 3: Linear Algebra / Group Theory (Matrix Transformations)

We encode the recurrence into a transformation matrix M. By computing M^n, we leverage structural symmetries to find F_n in $O(\log n)$ time, entirely bypassing sequential addition!

$$M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \quad M^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

Matrix Transformation Output ($M_{1,1}^{49}$): 12586269025

- **Mathematical Proof Aggregation**: true

```
begin
    function fib_matrix(n::Int=50)
        if n == 0 return Z(0) end

        # Define the base transformation matrix
        M = M_space([1 1; 1 0])

        # Exploit algebraic symmetry via matrix exponentiation
        M_n = M^(n-1)

        # F_n is located at the top-left (1,1) of M^(n-1)
        return M_n[1, 1]
    end

    ans_fib_matrix = fib_matrix(50)

    Markdown.parse("""
    **Matrix Transformation Output (\$M^{49}_{1,1}\$):**
    $(latexify("$(ans_fib_matrix)", env=:inline))

    * **Mathematical Proof Aggregation**: $(ans_fib_fp == ans_fib_aa ==
    ans_fib_matrix)
    """)
end
```